

Memory-as-a-Service (MaaS) for AI Agents — Backend Design Document

1. Project Overview

Memory-as-a-Service (MaaS) is a backend platform that provides persistent, queryable, and multi-scope memory for AI agents. It is designed as a production-style microservice that supports semantic memory, working memory, lifecycle controls, and multi-agent shared knowledge. The goal is to enable AI agents to store, retrieve, share, and resolve memory reliably across sessions and tasks, instead of relying on ad-hoc local memory hacks or prompt stuffing.

2. Objectives

- 1 Provide long-term semantic memory using embeddings
- 2 Provide short-term working memory with fast access
- 3 Expose retrieval APIs for agent memory search
- 4 Support TTL and lifecycle management
- 5 Support multi-agent shared memory
- 6 Provide conflict resolution strategies
- 7 Demonstrate backend system design maturity

3. Core Features

- 1 Memory write API
- 2 Semantic embedding and vector indexing
- 3 Hybrid retrieval (semantic + metadata + recency)
- 4 Working memory via Redis
- 5 Async embedding pipeline
- 6 Versioned memory records
- 7 Conflict detection and resolution
- 8 Memory decay and pruning
- 9 Agent, team, and org-level namespaces

4. System Architecture

Agents communicate with the MaaS API Gateway. Requests are routed to memory services. Metadata is stored in Postgres. Embeddings are stored in a vector database. Working memory is stored in Redis. Embedding jobs are processed asynchronously via a worker queue.

Agents → API Gateway → Memory Services
Memory Services → Postgres (metadata)
Memory Services → Vector DB (embeddings)
Memory Services → Redis (working memory)
Write → Queue → Embedding Worker → Vector DB

5. Technology Stack

- 1 Language: Python (FastAPI)
- 2 Metadata Store: PostgreSQL
- 3 Vector Store: Qdrant or pgvector
- 4 Working Memory: Redis
- 5 Async Jobs: Celery / RQ
- 6 Containerization: Docker
- 7 Observability: Prometheus + Grafana

6. Data Model

Memory Records Table

```
memory_records
id (UUID, PK)
org_id
agent_id
scope (agent/team/org/global)
content
memory_type
confidence
created_at
expires_at
version
source_agent
conflict_flag
embedding_id
usage_count
```

Memory Links Table (Optional Graph Layer)

```
memory_links
from_id
to_id
relation_type
weight
```

7. Memory Write Flow

```
POST /memory
→ validate request
→ store metadata row
→ enqueue embedding job
→ worker generates embedding
→ vector DB upsert
→ update embedding_id
```

8. Embedding Worker Design

Embedding worker consumes jobs, preprocesses text, chunks content, generates embeddings, stores vectors, and updates metadata. Includes retry logic and dead-letter queue.

9. Retrieval Pipeline

Query → embed query → vector search
→ metadata filter
→ hybrid ranking
→ return top results

Hybrid Ranking Formula

```
score =  
0.6 * semantic_similarity +  
0.2 * recency +  
0.1 * confidence +  
0.1 * usage_count
```

10. API Design

Write Memory

```
POST /memory  
{  
  content,  
  type,  
  scope,  
  ttl,  
  confidence  
}
```

Search Memory

```
POST /memory/search  
{  
  query,  
  scope,  
  filters,  
  top_k  
}
```

Working Memory

```
PUT /working/{session}  
GET /working/{session}
```

Conflict Resolution

```
POST /memory/resolve  
{  
  memory_ids,  
  strategy  
}
```

11. Conflict Resolution Strategies

- 1 Latest write wins
- 2 Confidence-weighted selection
- 3 Source trust weighting
- 4 Merge and flag conflict
- 5 Manual review required

12. TTL and Decay Engine

Supports hard TTL deletion and soft decay scoring. Soft decay uses confidence, recency, and usage frequency to gradually reduce memory priority before pruning.

```
decay_score =  
confidence * recency_factor * usage_factor
```

13. Multi-Agent Namespacing

- 1 Agent-level memory
- 2 Team-level shared memory
- 3 Organization-level memory
- 4 Global memory

14. Demo Prototypes

- 1 Customer support agent with preference memory
- 2 Planner + executor agents with shared plan memory
- 3 Debate agents with conflict resolution
- 4 Coding agent with learned bug memory

15. Observability

- 1 Memory write latency
- 2 Retrieval latency
- 3 Embedding queue lag
- 4 Memory hit rate
- 5 Conflict frequency

16. Deployment

- 1 Docker Compose with API, Postgres, Redis, Vector DB, Worker
- 2 Cloud deployment via Fly.io / Render / Railway
- 3 Managed vector DB option

17. Resume Value

This project demonstrates distributed backend design, hybrid storage, async processing, AI infrastructure patterns, and multi-tenant API architecture — making it a strong portfolio project for backend and AI-platform roles.